Image binarization by thresholding: Otsu's method

Laboratory 5, DEDP

Table of contents

1 Objective **1**

2 Theoretical aspects **1**

2.1 A grayscale image is a matrix 1

2.2 From pixels to a histogram 2

2.3 Thresholding and image binarization 3

2.4 Question: how to choose the threshold T? 3

2.5 Otsu's criterion: clusters should be compact 4

2.6 Summary: functions and notation you should know 4

3 Exercises **6**

1 Objective

Read and display grayscale images in Matlab, relate them to their intensity histograms, and implement image binarization using Otsu's threshold search and within-cluster variances.

2 Theoretical aspects

2.1 A grayscale image is a matrix

A digital image is a rectangular grid of **pixels**.

A grayscale image has one value per pixel, arranged in a matrix `I`.

Pixel values are numbers between 0 and 1 (type `double`) or between 0 and 255 (type `uint8`).

- 0 represents black;
- the maximum value (1 or 255) represents white;
- values in between represent shades of gray.

In this lab, we prefer pixel values to be in the range 0–1 and of type `double`, which allows for precise calculations.

For example, this creates and shows a small grayscale image:

```
I_small = [0 0.25 0.5 1; 0 0.25 0.5 1; 0 0.25 0.5 1]; % A 3-by-4 matrix of values between 0 a
imshow(I_small)
```

Images can be read from a file with the `imread()` function, as in the example below:

```
I = imread('coins.png');
% I = rgb2gray(I);  % If the image is RGB, convert it to grayscale first.
I = im2double(I);
imshow(I)
```

Functions used:

- `imread()` reads an image file and returns the matrix of pixel values;
- `rgb2gray()` converts an RGB image to grayscale, if needed;
- `im2double()` converts the image matrix to `double` and rescales its intensities to 0–1.
- `imshow()` receives the matrix and displays it as an image.

The images used in this lab are already grayscale, so we don't need to convert them from RGB to grayscale.

2.2 From pixels to a histogram

The image histogram counts how many pixels have each intensity, and shows the distribution of intensities.

Use `imhist()` directly on the image to display its intensity histogram:

```
figure
imhist(I)
xlabel('Intensity')
ylabel('Number of pixels')
```

For a grayscale image, `imhist(I)` uses 256 bins by default. Use `imhist(I,64)` to specify a different number of bins (here 64).

To transform the matrix into a vector, concatenating all the columns, use `I(:)`. This is useful for calculations that do not depend on pixel positions:

```
pixels = I(:);
```

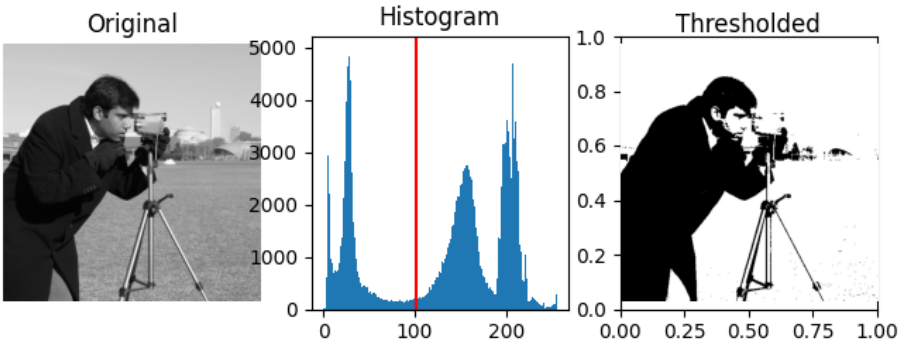


Figure 1: A real grayscale photograph, its intensity histogram, and its binarization. The red line marks the Otsu threshold in this published example.

2.3 Thresholding and image binarization

Image binarization converts a grayscale image into a binary image, with only two values (0=black and 1=white).

We can do this by **thresholding**, i.e. comparing with a threshold value T:

- all pixels $\leq T$ become 0
- all pixels $> T$ become 1

```
BW = I > 0.5;
figure
imshow(BW)
```

`BW` has the same rows and columns as `I`. Its values are `false` (= 0, displayed as black) and `true` (=1, displayed as white).

2.4 Question: how to choose the threshold T?

The question is how to choose a **good threshold** that separates the darker part of the image from the brighter part, not chosen by eye, but by an automatic criterion.

2.5 Otsu's criterion: clusters should be compact

Otsu's method chooses a threshold that minimizes the **weighted cluster variance**.

- for i = [1:254]:
  - compute the threshold $T = i/255$ (so that it has values in 0–1)
  - split the pixels into two clusters:
    - C_0 = pixels $\leq T$
    - C_1 = pixels $> T$
  - compute the number of pixels in each cluster, N_0 and N_1
  - compute the **mean** and **variance** of each cluster, m_0, m_1, v_0, v_1
  - compute the **weighted sum of variances**:

$$J = \frac{N_0}{N_0 + N_1} v_0 + \frac{N_1}{N_0 + N_1} v_1$$

  - save the score J for this threshold i in a vector `scores(i)`
- Pick the threshold that produced the **minimum score** in `scores`.

In Matlab, to compute the mean and variance of a vector `x`, use:

```
m = mean(x);
v = var(x,1);  % Use normalization by N here
```

If either of the two clusters is empty, skip that threshold and set its score J as `Inf` (infinity).

2.6 Summary: functions and notation you should know

Signature or notation	Example	Description
`imread(filename)`	`I = imread('coins.png')`	Read an image as a matrix of pixel values
`rgb2gray(I)`	`I = rgb2gray(I)`	Convert an RGB image to grayscale
`im2double(I)`	`I = im2double(I)`	Convert a `uint8` image to `double`, rescaling 0–255 to 0–1
`imshow(I)`	`imshow(I)`	Display an image I
`size(I)`, `numel(I)`	`size(I)`, `numel(I)`	Image dimensions and total number of pixels

Signature or notation	Example	Description
`I(:)`	`pixels = I(:)`	Transform a matrix into a column vector
`imhist(I)`, `imhist(I,n)`	`imhist(I,64)`	Display the intensity histogram; 256 bins by default for grayscale images
`I > T`	`BW = I > 0.5`	Binarize: pixels above the threshold become 1, the others 0
Logical indexing	`C0 = pixels(pixels <= T)`	Extract the values that meet a condition
`numel(x)`	`N0 = numel(C0)`	Count the pixels in a vector
`mean(x)`	`m0 = mean(C0)`	Mean intensity of a vector
`var(x,1)`	`v0 = var(C0,1)`	Variance, normalized by its number of pixels
`Inf(m,n)`	`scores = Inf(1,254)`	Initialize scores to infinity so skipped candidates cannot win
`min(x)`	`[minScore,index] = min(scores)`	Minimum value and the index of its first occurrence

3 Exercises

Write your commands in a script or live script, except for the function in Exercise 2, which goes in its own file.

1. Read, display, and binarize an image:
   a. Read `coins.png` into `I`, convert it to `double` and range 0–1, and display it.
   b. Display the image histogram. Which values correspond to the background and which to the coins?
   c. Choose a threshold T by inspecting the image and its histogram. Create the binary image `BW` via thresholding with T and display `BW`.
   d. Display the histogram of `BW`. Which two intensity values remain?
2. Implement Otsu's method as a function:
   a. Create the file `otsuManual.m` in the same folder as your script, with this header:

   ```
   function [bestThreshold] = otsuManual(I)
   ```

   The input `I` is a grayscale `double` image in 0–1. Return the best threshold found with the algorithm described above.
3. Apply your function:
   a. Call the function for the normalized image `I` from Exercise 1:

   ```
   bestThreshold = otsuManual(I);
   ```

   What is the value of `bestThreshold`?
   b. Binarize `I` with `bestThreshold` and display the result.
4. Repeat Exercise 3 for the image `Lab5_handwrittenText.png` available in the same folder.